

CS7DS2 Assignment - Week 8

Name: Kehan Liu ID:25334065

Q1

A

Based on the known objective function: $f(x_1, x_2) = (x_1 - 1.2)^2 + 2(x_2 - 2.5)^2 + 0.4x_1x_2$

To find the gradient of the original function, we first need to find the partial derivatives with respect to x_1 and x_2 .

Taking the partial derivative with respect to x_1

$$\frac{\partial f}{\partial x_1} = 2(x_1 - 1.2) + 0.4x_2$$

Taking the partial derivative with respect to x_2

$$\frac{\partial f}{\partial x_2} = 4(x_2 - 2.5) + 0.4x_1$$

Finally, the gradient vector of the original function is:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2(x_1 - 1.2) + 0.4x_2 \\ 4(x_2 - 2.5) + 0.4x_1 \end{bmatrix}$$

After obtaining the gradient vector, for the implementation of the projection gradient descent algorithm, I first perform gradient descent according to the basic gradient update formula

$x_{unproj}^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$, calculate the update point, and then project it onto the specified feasible region set.

```
def projected_gradient_descent ( x_init , projection_fn , lr = 0.1 , iterations = 60 ):  
    x = np . array ( x_init , dtype = float )  
    history_x = [ x.copy () ]  
    history_f = [ f ( x ) ]  
    for _ in range ( iterations ):  
        x_unproj = x - lr * grad_f ( x )  
        x = projection_fn ( x_unproj )  
        history_x . append ( x . copy () )  
        history_f.append ( f ( x ) )  
    return np . array ( history_x ) , np . array ( history_f )
```

B

For the feasible region $X_1 = \{(x_1, x_2) \in \mathbb{R}^2: 0.5 \leq x_1 \leq 2.5, 0.5 \leq x_2 \leq 3.5\}$, element-wise clipping is used to project onto X_1 . In the implementation, I use NumPy's clip function to directly restrict the horizontal coordinate $x[0]$ to the range of 0.5 to 2.5. If it is less than 0.5, it is calculated as 0.5, and if it is greater than 2.5, it is calculated as 2.5. Similarly, I restrict the vertical coordinate $x[1]$ to the range of 0.5 to 3.5. Finally, I repackage these two coordinates into a new point and return it.

```
def project_X1 ( x ):
    x1_proj = np . clip ( x [ 0 ], 0.5 , 2.5 )
    x2_proj = np . clip ( x [ 1 ], 0.5 , 3.5 )
    return np . array ( [ x1_proj , x2_proj ] )
```

This approach works because the feasible region X_1 forms a two-dimensional rectangular boundary parallel to the coordinate axes. Its constraints are completely independent in different dimensions. Finding the shortest distance projection from any point in space to this feasible region is mathematically equivalent to solving two independent one-dimensional problems.

For the orthogonal projection of a point in one-dimensional space onto a closed interval, the solution is to restrict the value of the point to the upper and lower bounds. Therefore, using the truncation operation independently for x_1 and x_2 is computationally efficient and results in the shortest distance.

C

The feasible region $X_2 = \{(x_1, x_2) \in \mathbb{R}^2: x_1 \geq 0.5, x_2 \geq 0.5, x_1 \leq x_2\}$ forms a polyhedron with sloping boundaries. Since the boundary lines $x_1 = 0.5$ and $x_1 = x_2$ are not orthogonal, using element-wise clipping cannot guarantee a correct projection.

Therefore, in the implementation, I utilized the geometric properties of this region for derivation. For any point outside the feasible region, its nearest effective projection point must lie on the two rays constituting the boundary of this region:

Boundary 1 (vertical line): $x_1 = 0.5$ and $x_2 \geq 0.5$. The projection point onto this ray is $p_1 = (0.5, \max(0.5, y_2))$.

Boundary 2 (diagonal): $x_1 = x_2$ and $x_1 \geq 0.5$. The projection point onto this ray is $p_2 = (\max(0.5, m), \max(0.5, m))$, where $m = \frac{y_1 + y_2}{2}$ is the average of the original coordinates.

The algorithm simply calculates the squared Euclidean distances from the original point to p_1 and p_2 , and returns the closer projection point.

```
def project_X2 ( x ):
    """
    Projection onto X2
    """
    y1 , y2 = x [ 0 ], x [ 1 ]
    # If the point is already within the feasible region, return it directly
    if y1 >= 0.5 and y2 >= y1 :
        return np.array ( [ y1 , y2 ] )
    # Calculate the projection onto boundary ray 1 (x1 = 0.5 and x2 >= 0.5)
    p1 = np . array ( [ 0.5 , max ( 0.5 , y2 ) ] )
    # Calculate the projection onto boundary ray 2 (x1 = x2 and x1 >= 0.5)
    m = ( y1 + y2 ) / 2.0
    p2 = np . array ( [ max ( 0.5 , m ) , max ( 0.5 , m ) ] )
    # Compare the distances from the two projection points to the original point (y1, y2)
    dist1 = ( p1 [ 0 ] - y1 ) ** 2 + ( p1 [ 1 ] - y2 ) ** 2
```

```
dist2 = ( p2 [ 0 ] - y1 ) ** 2 + ( p2 [ 1 ] - y2 ) ** 2
```

```
# Return the closer projection point
```

```
if dist1 < dist2 :
```

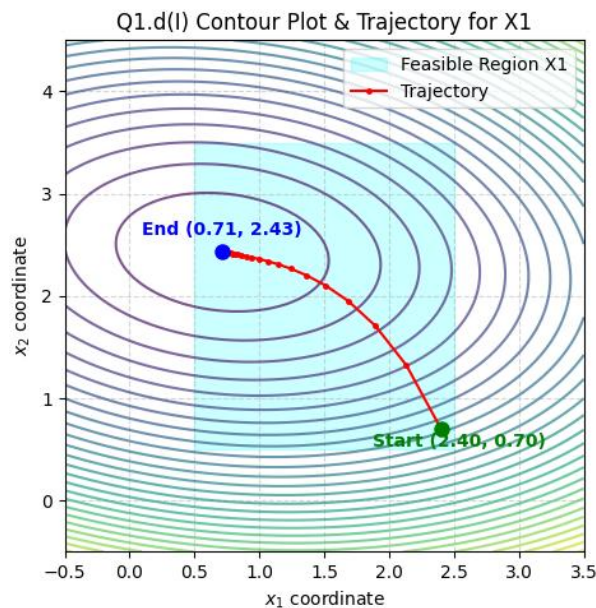
```
    return p1
```

```
else :
```

```
    return p2
```

D

To demonstrate the convergence behavior of the algorithm, the PGD algorithm was applied over the feasible region X_1 , starting from a given $x^{(0)} = (2.4, 0.7)$ point and performing 60 iterations. The visualization results are shown below.



Viewed from the contour lines, the function as a whole resembles an elliptical bowl. The function trajectory graph shows the algorithm starts from the initial point, which is already within the feasible region. The gradient descent throughout the process does not touch the boundaries; the trajectory is a smooth curve that crosses the contour lines and steadily descends towards the valley, without triggering any cutoff operations. The algorithm ultimately converges to the point at step $(0.71, 2.43)$. This point is not only X_1 the optimal solution for the function on, but also the globally unconstrained optimal solution.

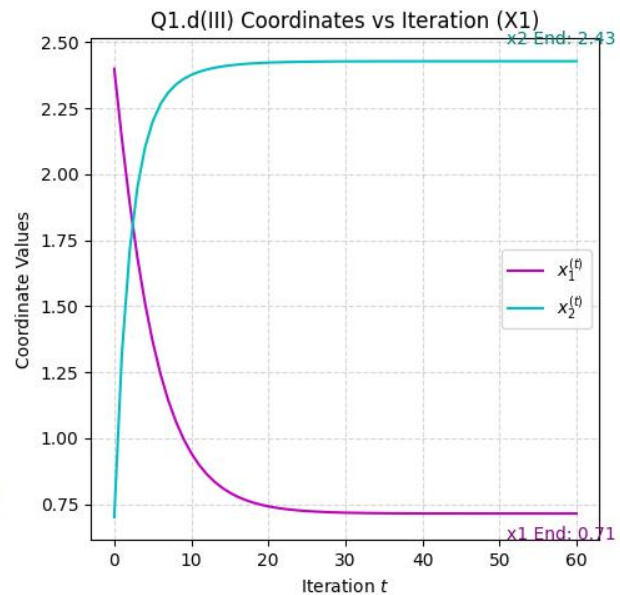
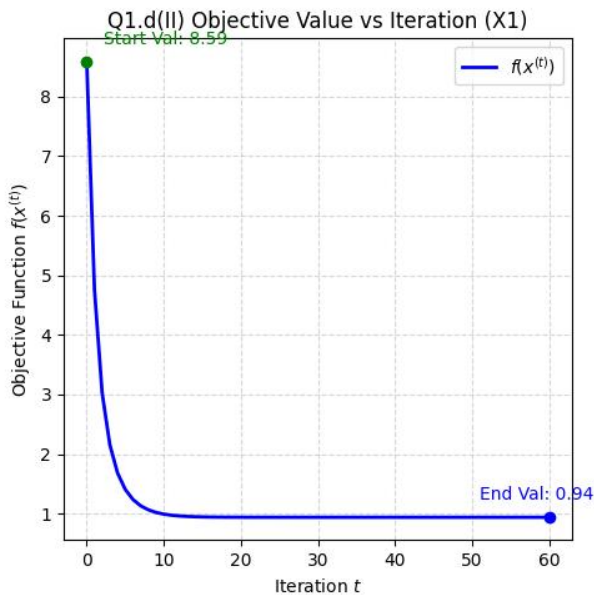


Figure (II): The descent curve of the objective function value shows how the objective function value $f(x^{(t)})$ changes with the number of iterations. Initially, the function value starts at a relatively high 8.59. In the first 10 iterations, due to the large gradient, the function value drops rapidly, reaching around 1 in the 10th iteration. After this, the curve becomes almost flat, indicating that the algorithm is very close to the minimum point and the gradient approaches zero. The final convergence value is 0.94, and the entire descent process is very smooth without any oscillations.

Figure (III): Coordinate value convergence curve. This figure records the changes x_1 in coordinates x_2 in each iteration.

The purple curve $x_1^{(t)}$ smoothly decreases from the initial value of 2.40 and eventually stabilizes at approximately 0.7.

The cyan curve $x_2^{(t)}$ rises rapidly from its initial value of 0.7 and eventually stabilizes at approximately 2.43.

The two curves show almost the same trend, and both tend to level off after about 20 iterations, proving that the PGD algorithm $\alpha = 0.1$ exhibits excellent convergence speed and stability on the objective function when using a constant step size.

E

Similarly, applying the PGD algorithm over the feasible region, the algorithm X_2 performed 60 iterations starting from a given point. The visualization results are shown below. $x^{(0)} = (2.4, 0.7)$

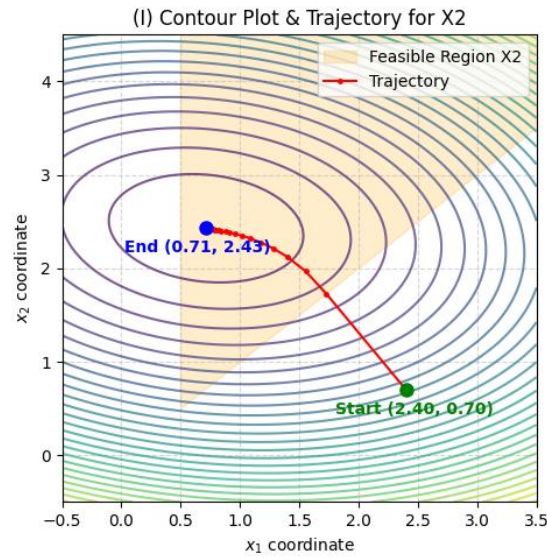


Figure (I): Contour lines and trajectory. This figure shows the trajectory of the algorithm in a feasible region with sloping boundaries X_2

As can be seen, unlike 1.d, the initial point is $x^{(0)} = (2.4, 0.7)$ located outside the feasible region X_2 . In the first step of the algorithm execution, the projection function takes effect, pulling the out-of-bounds point back with the shortest distance and projecting it onto the hypotenuse boundary $x_1 = x_2$.

After being pulled back to the boundary, guided by the gradient, the trajectory detaches from the boundary, enters the interior of the feasible region, and smoothly descends towards the valley. The algorithm eventually converges to the global optimum $(0.71, 2.43)$.

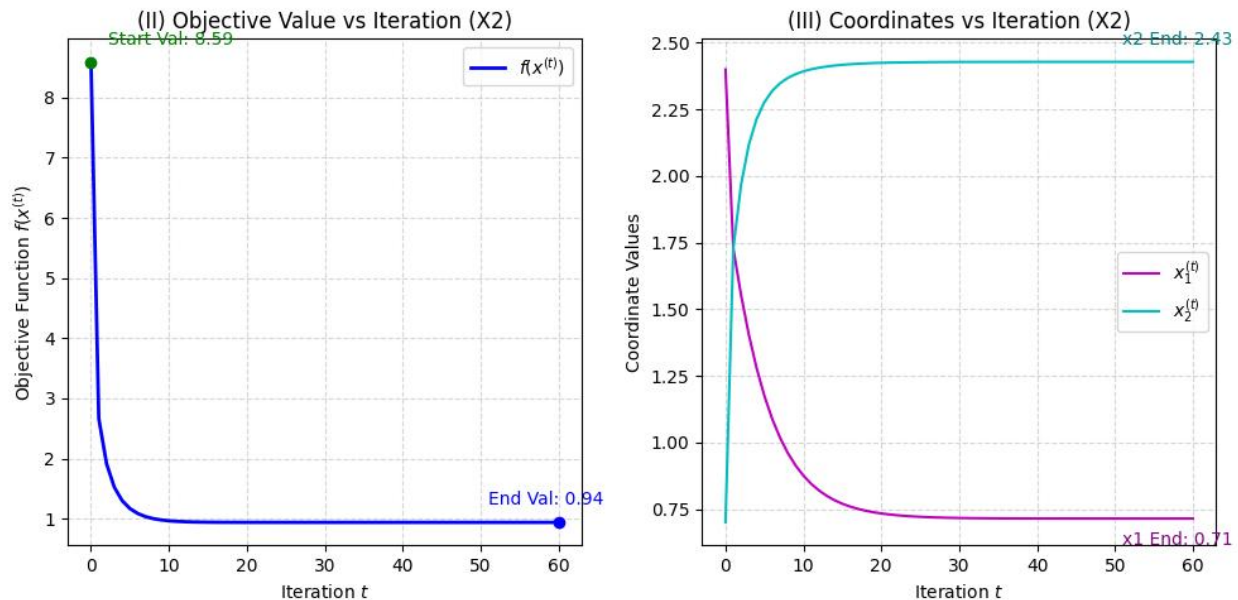


Figure (II): The objective function value descent curve shows the change of the objective function value $f(x(t))$ with the number of iterations. Initially, the function value starts at a relatively high 8.59. In the first 10 iterations, due to the large gradient, the function value drops rapidly, reaching around 1 in the 10th iteration. After this, the curve becomes almost flat, indicating that the algorithm is very close to the minimum point, and the gradient approaches zero. The final convergence value is 0.94, and the entire descent process is very smooth without any oscillations.

Figure (III): Coordinate value convergence curves. This figure records the changes of coordinates x_1 and x_2 in each iteration.

The purple curve $x_1(t)$ smoothly decreases from the initial value of 2.40 and eventually stabilizes at approximately 0.7.

The cyan curve $x_2(t)$ rises rapidly from the initial value of 0.7 and eventually stabilizes at approximately 2.43.

The two curves show almost the same trend, and both tend to level off after about 20 iterations, proving that the PGD algorithm exhibits excellent convergence speed and stability on the objective function when using a constant step size $\alpha=0.1$.

F

By solving for the gradient of the objective function $\nabla f(x) = 0$, we can obtain the global unconstrained minimum point of the function (0.714, 2.428).

Comparing the overall optimization results of the PGD algorithm on feasible regions X_1 and X_2 , the following conclusions can be drawn:

In the optimization tasks X_1 of and X_2 , the final iteration points all fall within the feasible region. This is because the global optimum (0.714, 2.428) satisfies both X_1 the constraints of and , and X_2 the algorithm eventually converges successfully to this internal global optimum.

In X_1 this case: the initial point (2.4, 0.7) lies inside the rectangular feasible region. Therefore, the entire optimization trajectory is as smooth as unconstrained gradient descent, and the boundary truncation operation is never triggered.

In X_2 the given condition: the initial point (2.4, 0.7) violates $x_1 \leq x_2$ the boundary constraints. Therefore, the algorithm is projected onto $x_1 = x_2$ the boundary during the first iteration. However, guided by the gradient, since the optimal point is located inside the region, the trajectory moves off the boundary in subsequent iterations and eventually converges to within the feasible region.

When comparing the objective function descent curves of X_1 and X_2 , a slightly faster convergence rate can be observed for X_2 . This is mainly because, in the case of X_2 , the initial point is not within the feasible region. The initial projection operation directly causes the point to make a large jump towards the optimum, saving some of the journey and speeding up the convergence

Q2

A

Based on the known objective function: $f(x_1, x_2) = (x_1 - 0.2)^2 + (x_2 - 2)^2$

To construct the penalty function, the original constraints ($x_1 \geq 0.5$ and $x_1 x_2 \geq 1$) must first be transformed into the standard form less than or equal to zero $g_i(x_1, x_2) \leq 0$:

For the first constraint $x_1 \geq 0.5$, rearranging the terms yields:

$$g_1(x_1, x_2) = 0.5 - x_1 \leq 0$$

For the second constraint $x_1 x_2 \geq 1$, rearranging the terms yields:

$$g_2(x_1, x_2) = 1 - x_1 x_2 \leq 0$$

Using $\max(0, g_i)$, we construct a function that penalizes only the case of constraint violation (i.e., when $g_i > 0$). Combining the penalty weights λ_1 and λ_2 , the linear penalty term Q is defined as:

$$Q(x_1, x_2) = \lambda_1 \max(0, 0.5 - x_1) + \lambda_2 \max(0, 1 - x_1 x_2)$$

The final augmented objective function to be optimized is $F(x_1, x_2)$. It is the sum of the original objective function and the penalty term:

$$F(x_1, x_2) = (x_1 - 0.2)^2 + (x_2 - 2)^2 + \lambda_1 \max(0, 0.5 - x_1) + \lambda_2 \max(0, 1 - x_1 x_2)$$

By minimizing $F(x_1, x_2)$, we seek the optimal solution that strikes a balance between reducing the original

B

This section implements a gradient descent algorithm based on fixed penalty weights.

Since the penalty term $Q(x)$ contains a function $\max(0, g_i(x))$ that is not differentiable at $g_i(x)=0$, I introduced subgradient logic in the code: when calculating the gradient in each iteration, the algorithm first evaluates whether the current point violates the constraint.

If $g_i(x) \leq 0$, i.e. the constraint is satisfied, then the contribution of the constraint to the gradient is 0.

If $g_i(x) > 0$, the constraint is violated, then the constraint is activated and the corresponding penalty gradient $\lambda_i \nabla g_i(x)$ is added.

The following is the Python implementation of the core algorithm logic:

```
def grad_F ( x , lam1 , lam2 ):
    gf = grad_f ( x )
    # Add the corresponding penalty gradient when the constraint is violated (g > 0)
    pen1 = lam1 * grad_g1 ( x ) if g1 ( x ) > 0 else np.array ( [ 0.0 , 0.0 ] )
    pen2 = lam2 * grad_g2 ( x ) if g2 ( x ) > 0 else np.array ( [ 0.0 , 0.0 ] )
    return gf + pen1 + pen2

# Gradient descent with fixed penalty method
def penalty_gradient_descent ( x_init , lam1 , lam2 , lr = 0.02 , iters = 100 ):
    x = np . array ( x_init , dtype = float )
    history_x = [ x.copy ( ) ]
    for _ in range ( iters ):
        gradient = grad_F ( x , lam1 , lam2 )
        x = x - lr * gradient
        history_x . append ( x . copy ( ) )
    return np.array ( history_x )
```

During the experiment, I used two penalty weights as required by the problem : one too small ($\lambda_1 = 0.5, \lambda_2 = 0.5$) and the other large enough ($\lambda_1 = 4.0, \lambda_2 = 4.0$). The results and visualizations will be reported in section q2.d.

C

The Primal-Dual method finds the optimum by alternately updating the original x and dual variables λ . Unlike the fixed penalty method in 2.b, the Lagrange multipliers in this algorithm λ_i are dynamically and adaptively adjusted.

Based on the formula given in the problem, the algorithm consists of two steps involving alternating iterations:

Primal Update: Perform gradient descent on the Lagrange function $L(x, \lambda)$. The penalty weights $\lambda^{(t)}$ are known constants at this point.

$$x^{(t+1)} = x^{(t)} - \alpha(\nabla f(x^{(t)}) + \lambda_1^{(t)} \nabla g_1(x^{(t)}) + \lambda_2^{(t)} \nabla g_2(x^{(t)}))$$

Dual Update: Performs gradient ascent on the Lagrange function λ and projects it back to the non-negative quadrant, which is represented by the $\max(0, \cdot)$ operation in the code.

$$\lambda_i^{(t+1)} = \max(0, \lambda_i^{(t)} + \beta g_i(x^{(t)}))$$

The following is the Python code implementation of this logic. In the experiment, we initialize the dual variable to $\lambda^{(0)} = [0, 0]$ and set appropriate step sizes α to ensure stable convergence of the algorithm.

```
def primal_dual_iteration ( x_init , alpha , beta , iters = 100 ):
    x = np . array ( x_init , dtype = float )
    # Initialize multipliers to 0
    lam = np.array ( [ 0.0 , 0.0 ] )
    history_x = [ x.copy ( ) ]
    history_lam = [ lam.copy ( ) ]
    for _ in range ( iters ):
        # Compute gradient of augmented Lagrangian wrt x
        grad_L_x = grad_f ( x ) + lam [ 0 ] * grad_g1 ( x ) + lam [ 1 ] * grad_g2 ( x )
        # Update x
        x_next = x - alpha * grad_L_x
        # Update multipliers lam
        lam1_next = max ( 0.0 , lam [ 0 ] + beta * g1 ( x ) )
        lam2_next = max ( 0.0 , lam [ 1 ] + beta * g2 ( x ) )
        lam_next = np . array ( [ lam1_next , lam2_next ] )
        x = x_next
        lam = lam_next
        history_x . append ( x . copy ( ) )
        history_lam . append ( lam . copy ( ) )

    return np . array ( history_x ) , np . array ( history_lam )
```

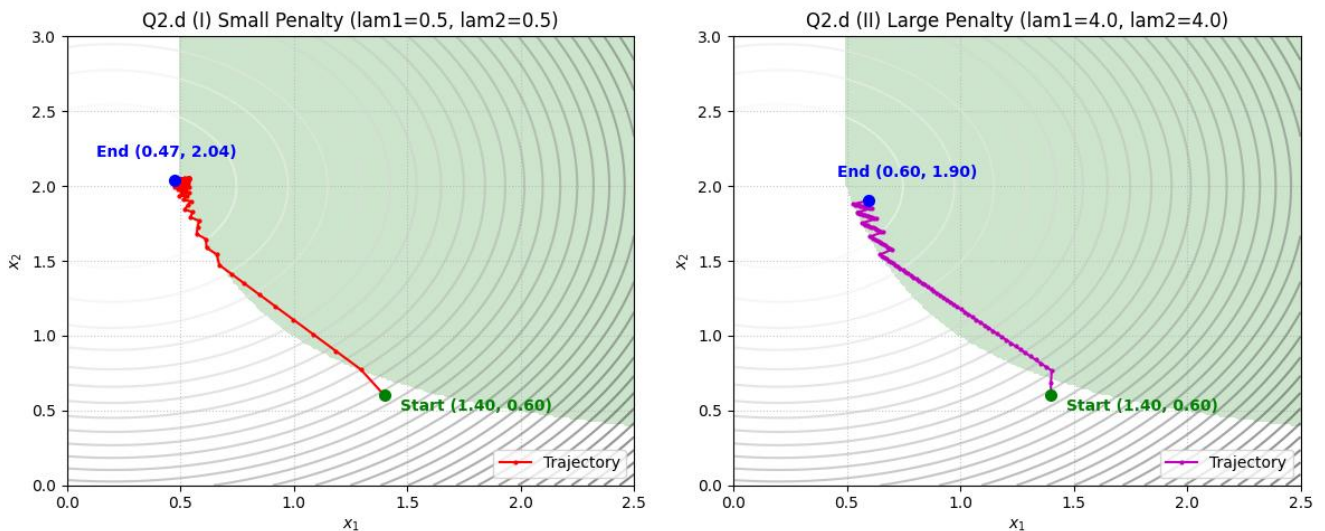
D

To illustrate the convergence behavior of the algorithm, a fixed penalty method was applied. The algorithm iterated 100 times from the initial point $x^{(0)} = (1.4, 0.6)$ with both small $\lambda = 0.5$ and large

penalty weights . The green shaded area in the figure represents the feasible region that satisfies all constraints. $\lambda = 4.0$

Before reporting the specific results, it's important to note that I used different iteration step sizes for these two penalty scenarios: a larger step size was used in the small penalty experiment $\alpha = 0.05$, while a smaller step size was strictly limited in the large penalty experiment $\alpha = 0.01$. I believe this is necessary because a huge penalty weight (e.g., $\lambda = 4.0$) causes the augmented objective function surface outside the constraint boundary to become extremely steep. In this extreme and sensitive numerical terrain, the penalty gradient is extremely large, and if a large step size is continued, even a slight deviation from the boundary will generate a huge reverse push. Therefore, to counteract the numerical instability caused by the strong penalty, the step size must be artificially reduced.

The final result is shown in the figure.



From the contour plot, similar to question q1, the function as a whole presents an elliptical bowl shape.

Figure (I) Small Penalty Case (Red Trajectory): This figure shows the running trajectory of the algorithm under small penalty weight ($\lambda = 0.5$). It can be seen that the algorithm starts from the initial point (1.4, 0.6), which is not within the feasible region. The penalty method has no projection mechanism, and the direction of movement of the iteration point in each step depends on the gradient ∇f and the penalty force $\lambda \nabla g$.

Since the initial point is in the penalty region, the penalty term is activated. However, because $\lambda = 0.5$ the weights are too small, the penalty force is very weak. In the first iteration, the gradient dominates. Therefore, the red trajectory does not rush to enter the feasible region at the beginning, but instead slopes to the upper left along the contour lines, continuing to move in the direction of the gradient.

Near the bottom, the function exhibits significant oscillations, increasing in frequency as it approaches the bottom, eventually stopping outside the green feasible region. This is because the true optimal solution (0.2, 2.0) lies outside the feasible region. When the penalty weight is insufficient, the benefit of decreasing the objective function by moving towards the optimal solution far outweighs the penalty for exceeding the boundary . Ultimately, it converges to a point that violates the constraints.

Figure (II) shows the case of a large penalty (purple trajectory): Starting from the same infeasible initial point, an extremely $\lambda = 4.0$ strong "penalty push" is generated. In the first few iterations, this push completely overwhelms the gradient, and the trajectory is almost a straight line, entering the feasible region via the shortest path. Only after entering the feasible region does it slide downwards along the gradient direction. It is worth noting that whenever the function touches the boundary, it is immediately bounced back almost vertically, eventually stopping at (0.61, 1.64), a value within the feasible region. This is because the true unconstrained minimum point is outside the wall, and the gradient of the objective function is constantly pulling the point out of the wall; once the point slightly goes out of bounds, the huge penalty bounces it back. In contrast to Figure 1, the penalty for going out of bounds far exceeds the benefit of the objective function descent brought by moving towards the optimal solution. Finally, it converges to a point within the feasible region.

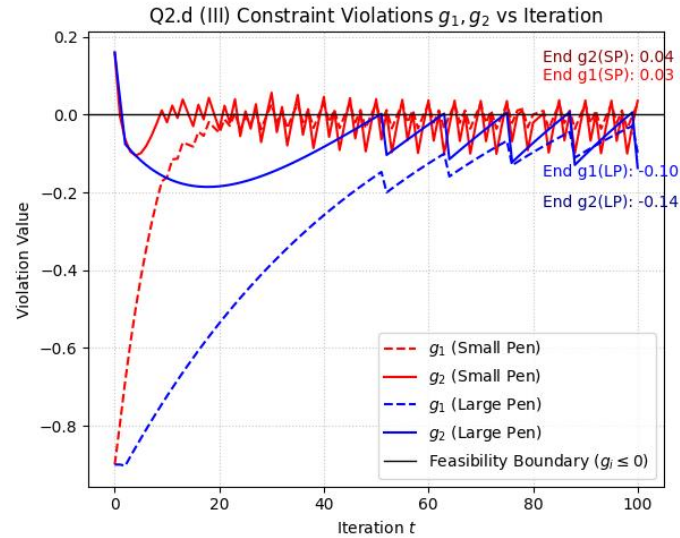


Figure (III) shows the trend of the values of the two inequality constraint functions $g_1(x(t))$ and $g_2(x(t))$ as the number of iterations changes under two penalty settings.

1. Minor penalty situations (red curve $\lambda_1 = 0.5, \lambda_2 = 0.5$):

Observing the red dashed line (g_1) and the red solid line (g_2) representing the weight of the small penalty, we can find that in the early stages of the iteration, the values of the two constraints quickly approach 0 and oscillate around this line. As the iteration progresses, they eventually both end up above the horizontal axis. Since both values are greater than 0, this indicates that the final convergence point remains in the infeasible region.

In the augmented objective function $F(x) = f(x) + Q(x)$, the penalty parameter λ was set too small, resulting in a gradient generated by the penalty term that was insufficient to counteract the gradient of the original objective function. Therefore, the algorithm compromised between reducing the objective function value and satisfying the constraints, ultimately failing to pull the point back into the feasible region.

2. Severe penalty situation (blue curve $\lambda_1 = 4.0, \lambda_2 = 4.0$):

In contrast, the blue dashed line (g_1) and the blue solid line (g_2) representing the large penalty weights rapidly approach the critical line ($y = 0$) after the first 50 iterations, and then stabilize at the critical line, oscillating at a frequency less than that of the red line. Finally, both g_1 and g_2 are less than 0, and the final convergence point remains within the feasible region.

This indicates that when the penalty parameter is large enough, any slight violation of the feasible region will be pulled back into the feasible region by the huge penalty value.

E

Based on the requirements of the problem, this section applies the original-dual iteration method to optimize the function. We set the update step size for the primal variable $\alpha = 0.06$ and the update step size for the dual multipliers $\beta = 0.08$, with the initial point set to $x^{(0)} = (1.4, 0.6)$ and the initial dual multipliers set to $(0, 0)$. The results are shown in the figure below.

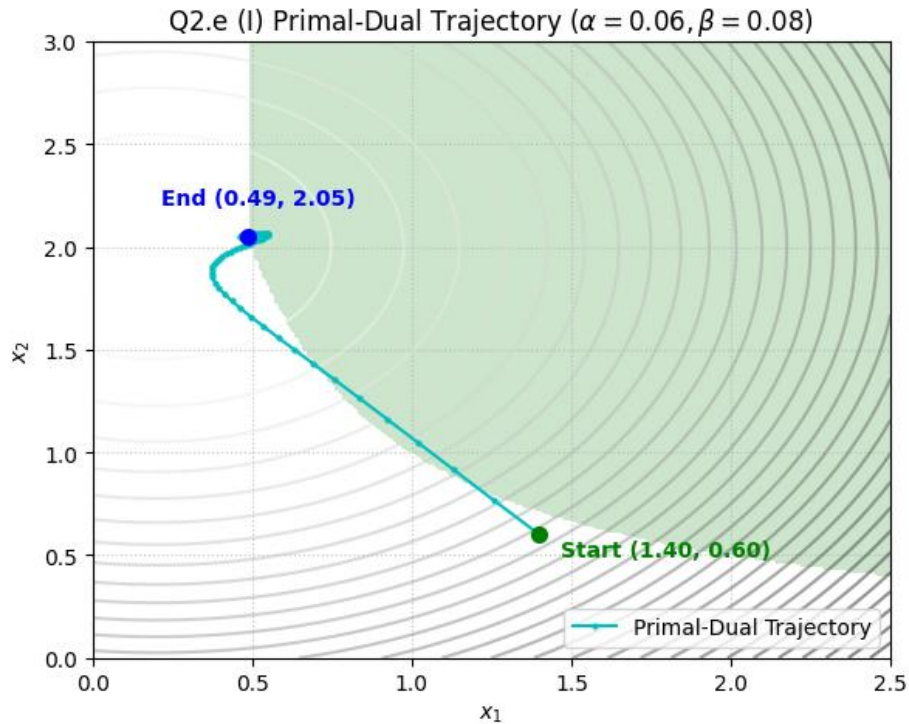


Figure (I): Observing the trajectory, it can be seen that after starting from the initial point, the algorithm moves almost in a straight line, smoothly to the upper left, without significant oscillations near the boundary or optimal solution, unlike the fixed penalty method. Instead, the trajectory eventually converges precisely and stops near the optimal solution (0.49, 2.05). This intuitively proves that the primal-dual algorithm can effectively find the optimal solution.

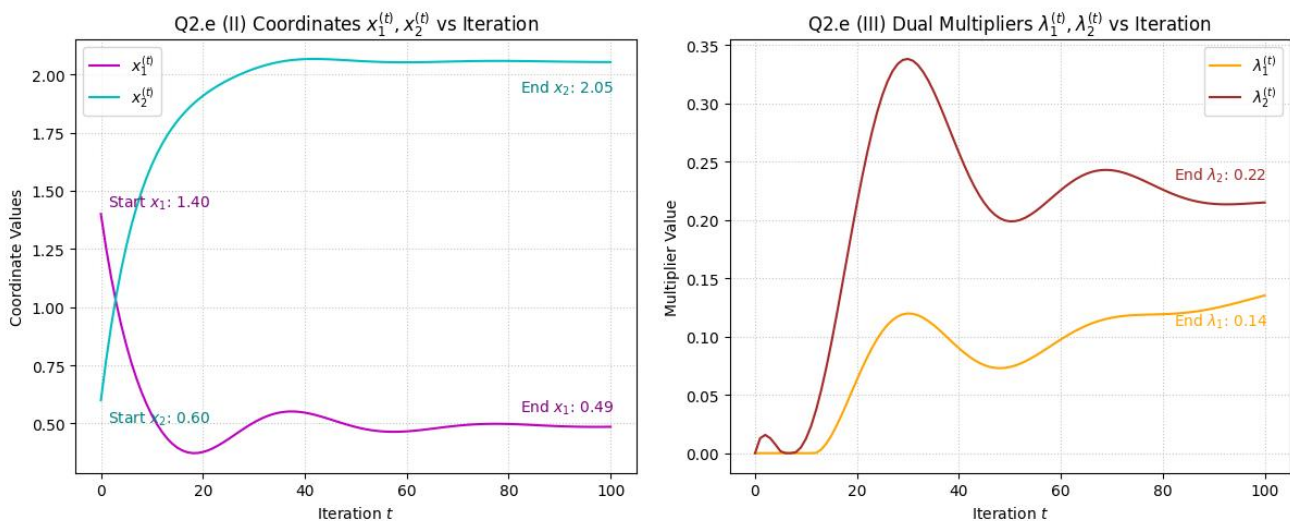


Figure 2: This figure illustrates the convergence process of the iterative variable in each dimension. representing the value x_1 , starts to drop rapidly from 1.4, initially falling below 0.5, then gradually decreasing in a wave-like pattern, eventually converging to 0.49.

represents x_2 a steady rise from 0.6, initially rising very rapidly, reaching approximately 2.0 after 40 iterations, then stabilizing and eventually converging to 2.05.

After undergoing rapid adjustments in the early stages, both curves exhibited extremely stable convergence in the mid-to-late stages, demonstrating that the selected step size $\alpha = 0.06$ has good numerical stability.

Figure 3: The variation of dual multipliers $\lambda_1^{(t)}, \lambda_2^{(t)}$ with the number of iterations.

In the early stages of the iteration, because the current x violates the constraints (i.e. $g_i(x) > 0$), the multipliers λ_1 (orange line) and λ_2 (brown line) accumulate rapidly during gradient updates. This is equivalent to the algorithm dynamically increasing the penalty, forcing the next x to move closer to the feasible region.

Once the iteration point is pulled back to the feasible region boundary and stabilizes, the constraint condition is satisfied $g_i(x) \leq 0$. At this point, the terms in the update rule $\beta g_i(x)$ become negative, causing the multipliers λ to begin to decrease. This decline phenomenon demonstrates the excellent adaptability of the original dual algorithm. When the violation is eliminated, the algorithm automatically weakens unnecessary penalties, thereby avoiding the boundary oscillations common in the fixed large penalty method.

As the iteration progresses into the middle and later stages, the variable x gradually approaches and converges to the optimal solution on the boundary. At this point, the constraint violation is minimal ($g_i(x) \approx 0$), and the update step size of the multipliers $\beta g_i(x)$ also approaches zero, causing λ_1 the sum to λ_2 stop fluctuating drastically.

Experimental results show that this method not only senses the degree of constraint violation by variables and spontaneously applies corrective force, but also eliminates redundant penalties through automatic callback of multipliers after the variables enter the feasible region. This game-theoretic mechanism enables the algorithm to robustly lock onto the optimal solution at the constraint boundary with minimal numerical error, demonstrating stability and performance far exceeding that of the fixed-penalty method.

F

feasibility :

Fixed penalty method: It is highly dependent on manually set penalty weights λ . Experiments show that when the weights are too small, the algorithm cannot effectively suppress the tendency of the objective function to descend into the infeasible region, leading to the failure of the final solution; while when the weights are too large, although feasibility can be guaranteed, it is often at the cost of numerical stability.

Primal-Dual Algorithm: This method exhibits greater intelligence. Through iterative updates of the dual variable, the penalty intensity spontaneously adjusts according to constraint violations. This mechanism ensures that after the iteration point falls into the feasible region, the multipliers remain at a level that precisely cancels out the target gradient, thus locking in the solution more accurately.

Convergence and numerical stability

Fixed penalty method: has obvious drawbacks. The large λ value set to satisfy hard constraints often produces huge jumps near the boundary, causing violent oscillations at the iteration point. It usually requires a very small learning rate to converge slowly, resulting in low efficiency.

The primal-dual algorithm exhibits superior convergence properties. Because the multipliers λ evolve smoothly, they provide a gradual guiding force for the original variables. Experimental curves show that after a rapid initial adjustment, the coordinate and multiplier changes in the mid-to-late stages of the primal-dual method are extremely stable, demonstrating that the algorithm possesses stronger numerical stability and convergence quality under complex constraints.

While the fixed penalty method is simple to implement, it is difficult to tune parameters and has poor robustness. In contrast, the original dual iteration method introduces an adaptive dual variable game, which significantly improves the convergence accuracy and robustness of the algorithm while ensuring the strict feasibility of the solution. It is a more ideal choice for solving such constrained optimization problems.

Q3

A

The feasible region X is defined by the following five linear inequality constraints:

$$(1) x_1 \geq 0.5 \quad (2) x_2 \geq 0.5 \quad (3) x_1 \leq 3 \quad (4) x_2 \leq 3 \quad (5) x_1 + x_2 \leq 4$$

From a geometric perspective, constraints (1) to (4) define a rectangular region $[0.5, 3] \times [0.5, 3]$. Constraint (5) is a straight line with a slope of -1 that cuts off the upper right corner of this rectangular region. Therefore, the feasible region X is a convex pentagon.

By finding the intersection of the boundary lines of the above inequality, we can determine the coordinates of the five vertices of the pentagon:

V_1 (Bottom left corner): Intersection (1) & (2) $\Rightarrow (0.5, 0.5)$. Satisfies $0.5 + 0.5 = 1 \leq 4$, valid.

V_2 (Top left corner): Intersection (1) & (4) $\Rightarrow (0.5, 3.0)$. Satisfies $0.5 + 3.0 = 3.5 \leq 4$, valid.

V_3 (Top turning point): Intersection (4) & (5) $\Rightarrow x_2 = 3, x_1 + 3 = 4 \Rightarrow (1.0, 3.0)$.

V_4 (Right-side turning point): Intersection (3) & (5) $\Rightarrow x_1 = 3, 3 + x_2 = 4 \Rightarrow (3.0, 1.0)$.

V_5 (Bottom right corner): Intersection (3) & (2) $\Rightarrow (3.0, 0.5)$. Satisfies $3.0 + 0.5 = 3.5 \leq 4$, valid.

In summary, the set of vertices of the feasible region is:

$$V = (0.5, 0.5), (0.5, 3.0), (1.0, 3.0), (3.0, 1.0), (3.0, 0.5).$$

In each iteration of the Frank-Wolfe algorithm, we need to solve the following linear subproblem:

$$\min_{s \in X} \langle \nabla f(x^{(t)}), s \rangle$$

Here, $x^{(t)}$ is the current iteration point, $\nabla f(x^{(t)})$ which is a constant vector in this step.

The objective function of this subproblem, $L(s) = \nabla f(x^{(t)})^T s$, is linear with respect to s . According to the fundamental theorem of linear programming, the extreme values of a linear function over a compact convex set are always attained at the extreme points of that set. For a convex polytope such as our feasible region X , these extreme points correspond to its vertices. Since the optimal movement direction always points towards the most distant part of the feasible region in the direction of the negative gradient $-\nabla f(x^{(t)})$, the optimal solution must lie on the boundary. Because the boundary of the feasible region X is composed of linear segments, the optimal solution is

necessarily obtained at a vertex or along an entire edge containing that vertex. This property significantly simplifies the implementation, as the algorithm only needs to search through the five identified vertices to find the point that minimizes the linear objective.

B

The objective function is known to be: $f(x_1, x_2) = (x_1 - 1.5)^2 + (x_2 - 1.2)^2$

To obtain the gradient vector $\nabla f(x_1, x_2)$, we need to take the partial x_1 derivatives with respect to the variables and x_2

Taking the partial derivative with respect to x_1 : $\frac{\partial f}{\partial x_1} = 2(x_1 - 1.5)$

Taking the partial derivative with respect to x_2 : $\frac{\partial f}{\partial x_2} = 2(x_2 - 1.2)$

The gradient vector of the objective function $\nabla f(x_1, x_2)$ is: $\nabla f(x_1, x_2) = [2(x_1 - 1.5), 2(x_2 - 1.2)]$

C

In the implementation of this problem, the algorithm follows the following iterative logic:

First, calculate the gradient of the objective function $\nabla f(x^{(t)}) = [2(x_1 - 1.5), 2(x_2 - 1.2)]^T$.

Next, solve the linear subproblem to find a vertex $z^{(t)} \in X$ whose inner product with the gradient is minimized:

$$\min_{z \in V_1, \dots, V_5} \langle \nabla f(x^{(t)}), z \rangle$$

Based on the derivation in 3.a, we only need to traverse the five identified vertices to find the optimal direction.

Using the fixed step sizes (0.8 and 0.95) given in the problem γ , update the iteration points according to the convex combination formula.

$$x^{(t+1)} = (1 - \gamma)x^{(t)} + \gamma z^{(t)}$$

In the Python implementation, we define a vertex matrix `VERTICES` and find the target vertex using `np.dot` in each iteration. The core algorithm functions are as follows:

```
def frank_wolfe ( x_init , gamma , iters = 60 ):
    x = np . array ( x_init , dtype = float )
    history_x = [ x.copy ( ) ]
    history_z = [ x.copy ( ) ]
    history_f = [ f_q3 ( x ) ]

    for _ in range ( iters ):
        gradient = grad_f_q3 ( x )
        # Solve linear subproblem: find the vertex with minimal inner product by iteration
        best_val = np.inf
        best_z = None
        for v in VERTICES:
            val = np.dot ( gradient , v )
            if val < best_val :
                best_val = val
```



```

best_z = v

# FW update step
z = best_z

x = x + gamma * ( z - x )

history_x . append ( x . copy() )
history_z . append ( z . copy() )
history_f . append ( f_q3 ( x ) )

return np . array ( history_x ), np . array ( history_z ), np . array ( history_f )

```

As required by the problem, the algorithm $x^{(0)} = (2.8, 0.8)$ starts execution from the initial point. We conducted 60 iterations for each step size $\gamma = 0.8$ and $\gamma = 0.95$ to observe their impact on the convergence trajectory and the descent of the objective function.

D

As required by the experiment, we used an initial point $x^{(0)} = (2.8, 0.8)$ to test two large fixed step size parameters: $\gamma = 0.8$ and, respectively $\gamma = 0.95$, and the results are shown below.

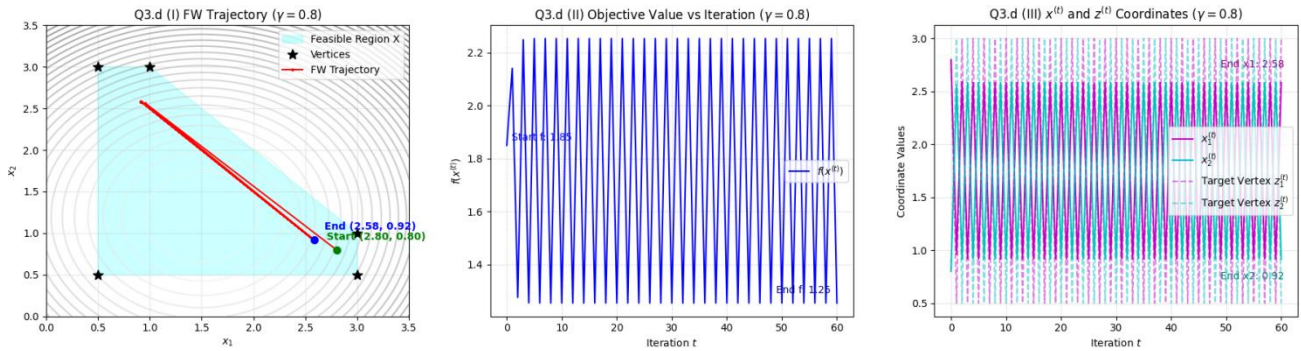
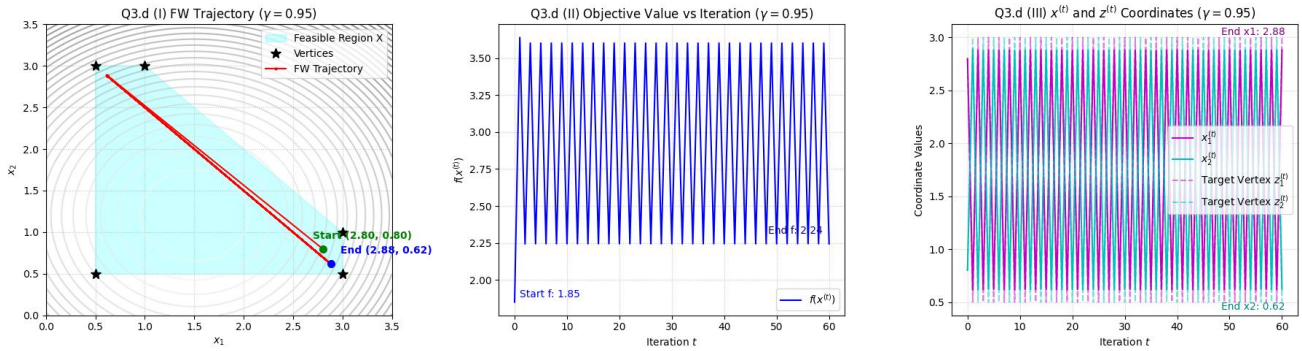


Figure (I) Trajectory Diagram: Observing the trajectory in the figure, it can be seen that the iteration points do not gradually move closer to the theoretical optimal solution. Instead, the trajectory makes high-frequency, large-amplitude back-and-forth jumps between the upper left vertex and the right boundary of the feasible region. I believe this is due to severe overshoot caused by an excessively large step size γ in the update formula $x^{(t+1)} = (1 - \gamma)x^{(t)} + \gamma z^{(t)}$. According to the update formula, when $\gamma = 0.8$, the new iteration point would be strongly pulled towards the currently selected target vertex $z(t)$. This caused the algorithm to skip the optimal point in one step, directly rushing to the other side of the feasible region.

Figure (II) Objective function value: In a normal gradient descent process, the objective function value $f(x^{(t)})$ should decrease monotonically or approximately monotonically. However, the curve in Figure (II) exhibits unstable sawtooth fluctuations, with the function value repeatedly jumping between lower and higher points, showing no signs of convergence. This corresponds precisely to the result in Figure 1. When the iteration point overshoots the central region where the optimal solution is located, the function value increases sharply; at this point, the gradient direction reverses drastically, guiding the algorithm to jump to the opposite vertex in the next step, thus falling into an infinite loop.

Figure (III) Coordinates $x^{(t)}$ and Target Vertex $z^{(t)}$ This figure reveals the cause of the oscillation from the perspective of coordinate changes . The dashed line representing the target vertex undergoes a vertical numerical abrupt change after each iteration , proving that the gradient direction reverses by 180 degrees after each step. Due to γ its large size, the actual iterative coordinates $x^{(t)}$ are pulled by the target vertex, also jumping up and down repeatedly .



As can be seen, $\gamma = 0.95$ the overall pattern of the three images $\gamma = 0$ is not significantly different from that of $\gamma = 0.8$ case. The only difference is that, due to the step size being closer to 1, the algorithm experiences a larger overshoot between vertices, resulting in a higher peak value for the objective function. Furthermore, the actual coordinate variables $x^{(t)}$ are almost entirely $z^{(t)}$ dominated by the target vertex, with their trajectories highly overlapping. This indicates that a fixed large step size completely eliminates the algorithm's convergence ability near the minimum point .

Based on this, we can conclude that in the Frank-Wolfe algorithm, an excessively large fixed step size leads to severe overshoot. The algorithm not only fails to converge but also becomes mired in violent periodic oscillations between the boundary vertices of the feasible region. This indicates that to make the algorithm effectively approach the optimal solution, the magnitude of its movement towards the target vertex in a single iteration must be limited. To verify this conclusion, I supplemented this with a set of $\gamma = 0.1$ control experiments using a smaller fixed step size (). The results are as follows.

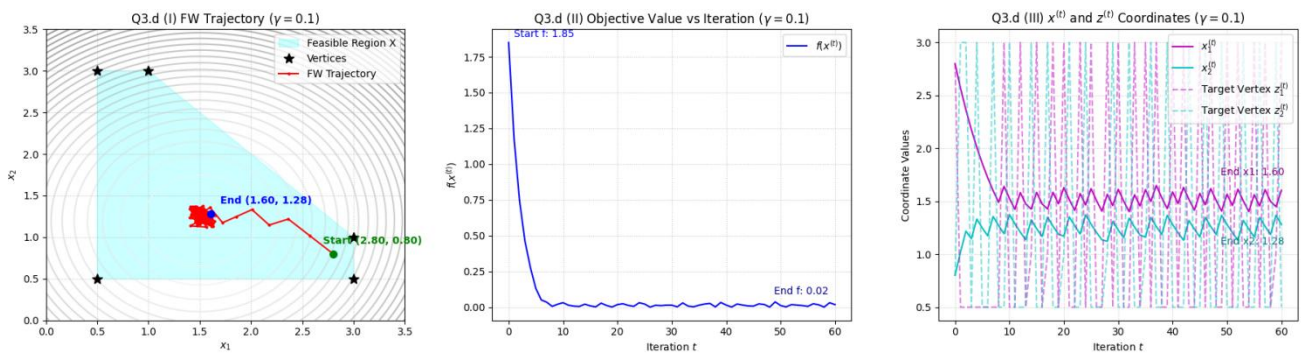


Figure (I) Trajectory Graph: Unlike the full-screen horizontal jumps under large step sizes, the red trajectory now starts very smoothly from the starting point (2.8, 0.8) and steadily approaches the central region where the theoretical optimal solution (1.5, 1.2) is located. Close observation reveals that the trajectory exhibits a zig-zag shape. This is a typical characteristic of the FW algorithm ; since the algorithm can only move along the vertex direction, it alternately adjusts its direction by continuously switching target vertices to approach the center of the circle.

Figure (II) shows the objective function value: the objective function value did not exhibit sawtooth fluctuations. The curve dropped rapidly in the first 10 iterations , then converged smoothly to near the minimum and remained stable , eventually settling at 0.02, very close to the optimal solution.

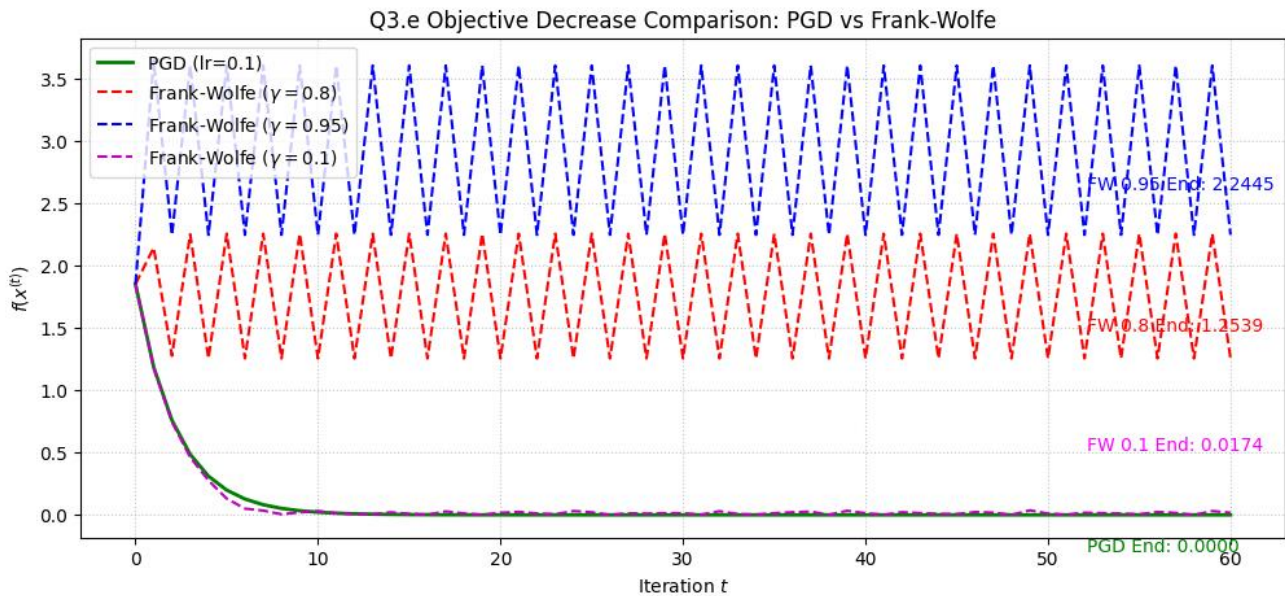
This demonstrates that the smaller step size successfully suppressed overshoot, allowing each iteration to effectively reduce the numerical value .

Figure (III) shows the coordinate changes: The dashed line representing the target vertex $z^{(t)}$ continues to jump drastically up and down . This is mainly because once the optimal solution is crossed, the gradient direction reverses and points towards the opposite vertex. However, observing the $x^{(t)}$ solid lines representing the actual coordinates, they converge rapidly in the first ten steps, x_1 steadily decreasing to around 1.5 and then x_2 increasing to around 1.2 . Subsequently, they oscillate around this value. I believe this is because while a fixed, small step size avoids overshoot in the initial stages, when the iteration point $x^{(t)}$ approaches the center, the distance from the current point to the target vertex $z^{(t)}$ is still large. At this point, the absolute displacement generated by each update $\gamma(z^{(t)} - x^{(t)})$ is greater than the distance from the current point to the true optimal solution. Therefore, $x^{(t)}$ can only jump around the optimal solution .

By comparing large and small step sizes, it can be shown that the convergence quality of the Frank-Wolfe algorithm depends on the choice of step size. Although using a sufficiently small fixed step size can restore the convergence of the algorithm, this inevitably increases the number of iterations required for convergence.

E

To compare its performance with the Frank-Wolfe algorithm, this section implements the Projected Gradient Descent (PGD) algorithm for the same objective function and feasible region. The implementation logic is consistent with Q1 and will not be described in detail here. The PGD algorithm uses the exact same initial point as FW $x^{(0)} = (2.8, 0.8)$ and sets a constant learning rate $lr = 0.1$, with the same number of iterations of 60. The results are as follows.



Frank-Wolfe ($\gamma = 0.8$ & $\gamma = 0.95$): As analyzed in Section 3.d, the FW algorithm failed to converge under both parameter settings due to the excessively large step size. The objective function value exhibited typical oscillating behavior in the range far above the theoretical optimal value, with the final values settling at 1.25 and 2.24 respectively.

Frank-Wolfe ($\gamma = 0.1$): This purple line is the best performing line in the FW algorithm . Compared to the red and blue lines, it does not exhibit oscillations, and the objective function value decreases smoothly. The rate of decrease is basically equal to that of PGD, but in the later stages of iteration, due to the limitation of convex combinations between vertices, it will experience slight oscillations

when approaching the optimal solution, corresponding to the small fluctuations of the purple line in the later stages of the graph.

PGD($\text{lr} = 0.1$): In comparison, the green PGD curve performs best. The objective function value starts from the initial state and decreases rapidly along an extremely smooth curve. After about 10 iterations, the curve reaches the optimal solution and remains at that point thereafter.

The comparison demonstrates that the performance of the FW algorithm is far inferior to that of the PGD algorithm. PGD is superior in both convergence speed and stability.

F

Boundary Behaviour

The FW algorithm, because the optimal solution to its linear subproblems always lies on a vertex of the feasible region, makes its search direction highly dependent on the boundaries. In this problem, although the global optimal solution lies inside the feasible region, the FW algorithm still shuttles back and forth between boundary vertices, exhibiting strong boundary dependence.

The PGD algorithm exhibits more flexible boundary behavior. When an iteration point crosses the boundary, the projection operation pulls it back to the nearest boundary; however, once the iteration point enters the feasible region, the algorithm becomes pure gradient descent, independent of the boundary.

Oscillation

The FW algorithm is highly prone to oscillations. Experiments have shown that when using a fixed step size (such as $\gamma = 0.8, 0.95$ even 0.1), the iteration point repeatedly jumps between vertices on both sides of the optimal solution because the algorithm always moves towards the farthest vertex. Even with an extremely small step size, it is impossible to eliminate the oscillations near the optimal solution.

PGD: When the learning rate is chosen appropriately, PGD exhibits excellent numerical stability, with a smooth trajectory and no oscillations.

Objective Decrease

Due to the oscillation phenomenon of FW, the objective function value of FW fluctuates violently in a "sawtooth" pattern under large step sizes, and the algorithm cannot converge. Under small step sizes, its descent speed is close to that of PGD, and the descent speed is acceptable.

PGD: The objective function value of PGD exhibits a smooth decrease. It is faster and more stable. Thanks to the guidance of the gradient direction, the algorithm can quickly approach the center, reaching the optimal solution in the first few iterations and maintaining it thereafter.

Ease of Implementation

The implementation of FW is simpler. To implement FW, you only need to solve a very simple linear programming subproblem. It does not involve any complex geometric projection calculations. The code is very intuitive and less prone to errors.

PGD is more difficult to implement than FW. To project out-of-bounds points back into the feasible region of the polygon, the boundary conditions must be analyzed and the decision logic written first. This can be very troublesome for more complex boundary conditions.

For this problem, the PGD algorithm outperforms the Frank-Wolfe algorithm in terms of convergence, stability, and accuracy. However, the Frank-Wolfe algorithm avoids projection calculations because it does not need to consider complex boundary conditions, making it simpler

to implement when facing extremely difficult projection problems. Moreover, with the right step size, its performance is close to that of PGD.